

Báo cáo bài thực hành Buổi 1

Ảo hóa cho hệ thống IoT với Docker, Mosquitto và Python

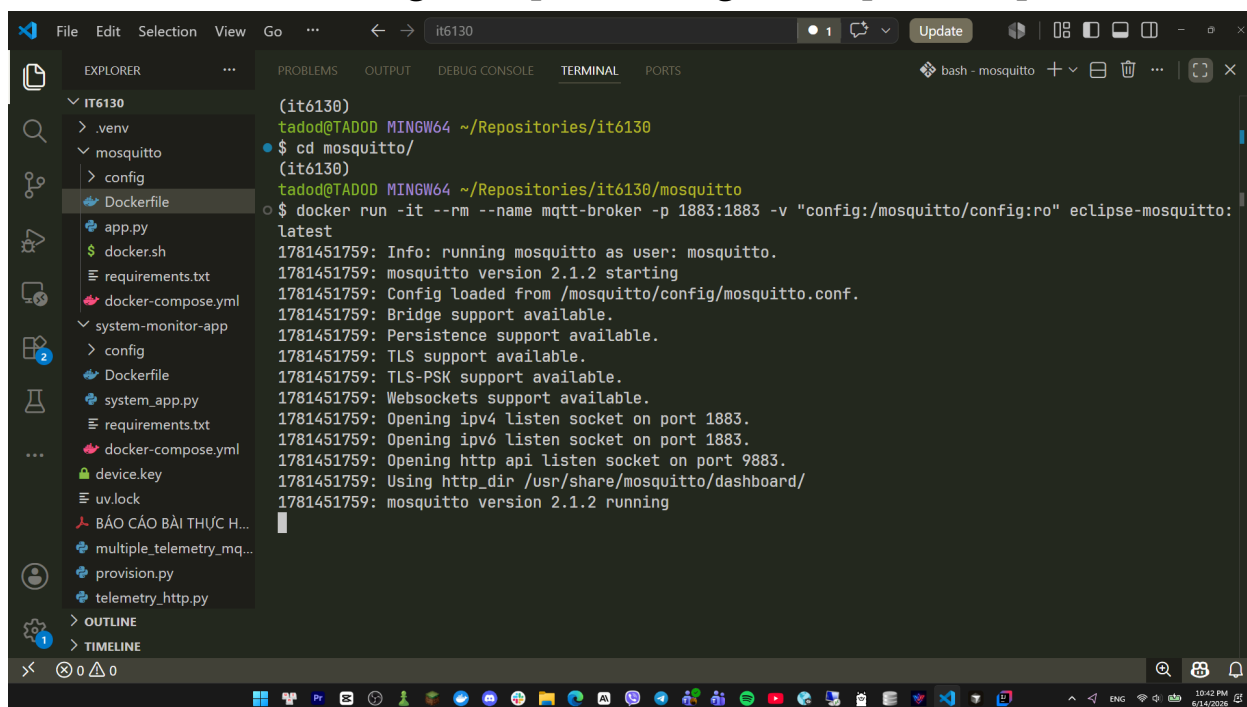
Sinh viên thực hiện: Đỗ Thành Đạt - 20261103M

MỤC LỤC

1. Chạy thành công MQTT broker bằng Docker 1
2. Kiểm thử publish/subscribe bằng mosquitto_pub và mosquitto_sub 2
3. Chạy, hiểu và kiểm thử chương trình app.py được cung cấp sẵn 3
4. Container hóa chương trình app.py bằng Dockerfile 5
5. Chạy ít nhất 3 sensor simulator container 5
6. Subscribe topic iot/sensors và chụp màn hình dữ liệu nhận được 7
7. Tự viết một ứng dụng IoT riêng, ví dụ ứng dụng gửi thông tin CPU/RAM lên topic system/info 8
8. Tự docker hóa ứng dụng IoT riêng đó và chạy trong Docker network iot-net 10
9. Subscribe topic riêng của ứng dụng mới và chụp màn hình dữ liệu nhận được 12
10. Trả lời các câu hỏi ở Mục 14 13

1. Chạy thành công MQTT broker bằng Docker

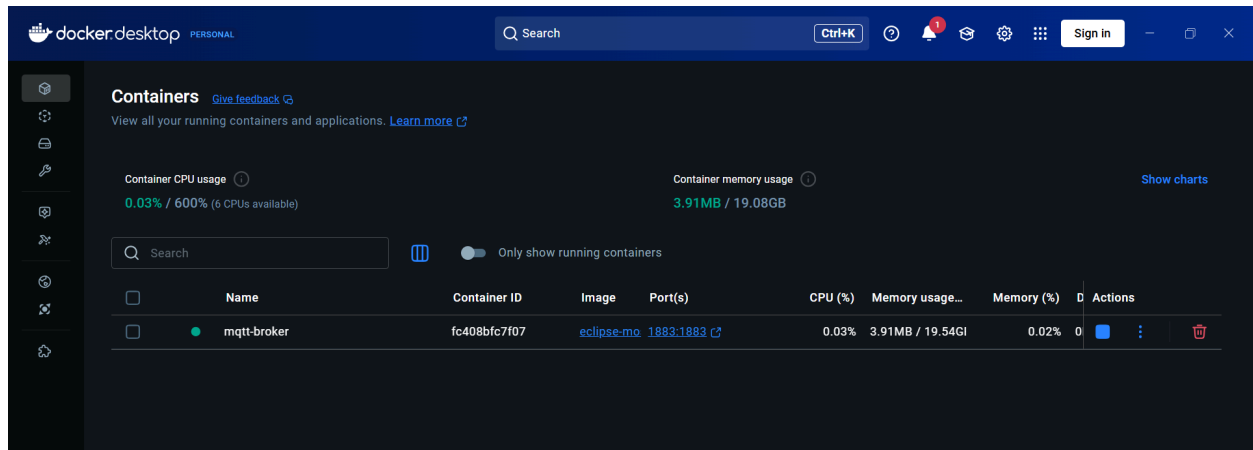
- Chạy bằng terminal: `docker run -it --rm --name mqtt-broker -p 1883:1883 -v "config:/mosquitto/config:ro" eclipse-mosquitto:latest`



The screenshot shows a VS Code editor with a terminal window open. The terminal is running a command to start an MQTT broker using Docker. The command is: `docker run -it --rm --name mqtt-broker -p 1883:1883 -v "config:/mosquitto/config:ro" eclipse-mosquitto:latest`. The output shows the broker starting successfully, displaying version 2.1.2 and various supported features like bridge, persistence, TLS, and websockets. The terminal also shows the file explorer on the left with a project structure including a `config` directory and a `Dockerfile`.

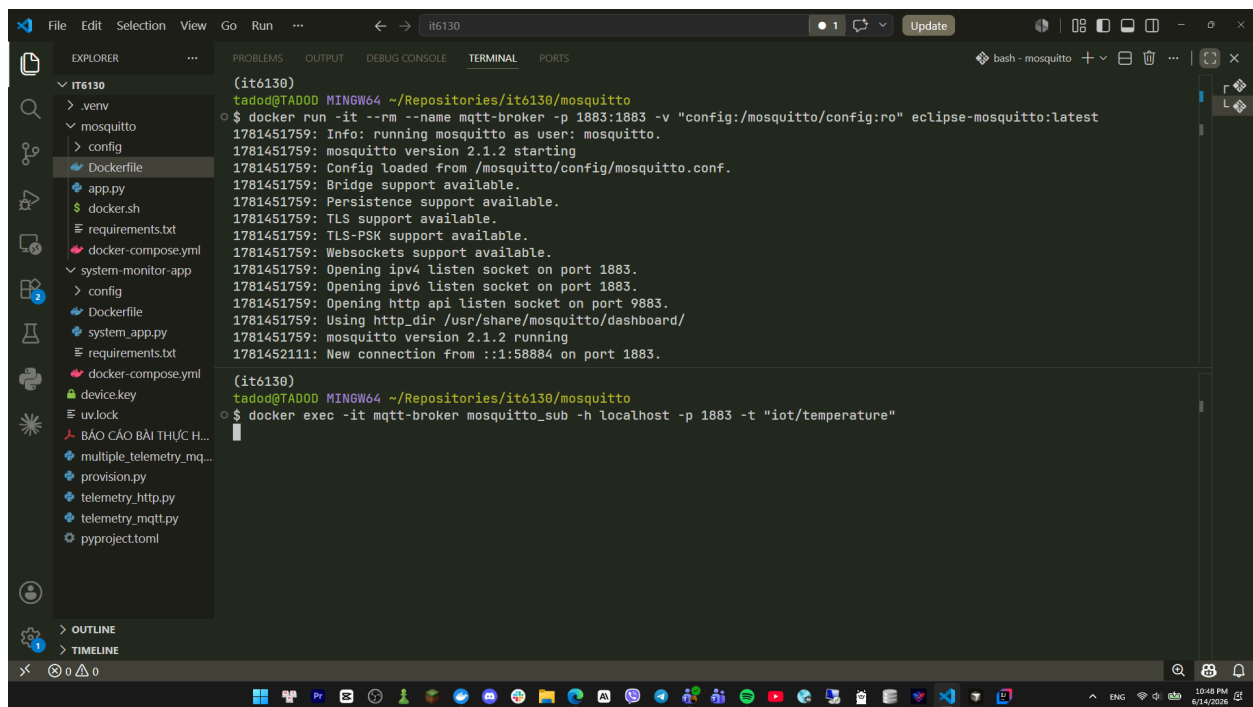
```
(it6130)
tadod@TADOD MINGW64 ~/Repositories/it6130
$ cd mosquitto/
(it6130)
tadod@TADOD MINGW64 ~/Repositories/it6130/mosquitto
$ docker run -it --rm --name mqtt-broker -p 1883:1883 -v "config:/mosquitto/config:ro" eclipse-mosquitto:latest
1781451759: Info: running mosquitto as user: mosquitto.
1781451759: mosquitto version 2.1.2 starting
1781451759: Config loaded from /mosquitto/config/mosquitto.conf.
1781451759: Bridge support available.
1781451759: Persistence support available.
1781451759: TLS support available.
1781451759: TLS-PSK support available.
1781451759: Websockets support available.
1781451759: Opening ipv4 listen socket on port 1883.
1781451759: Opening ipv6 listen socket on port 1883.
1781451759: Opening http api listen socket on port 9883.
1781451759: Using http_dir /usr/share/mosquitto/dashboard/
1781451759: mosquitto version 2.1.2 running
```

- Trên Docker Desktop

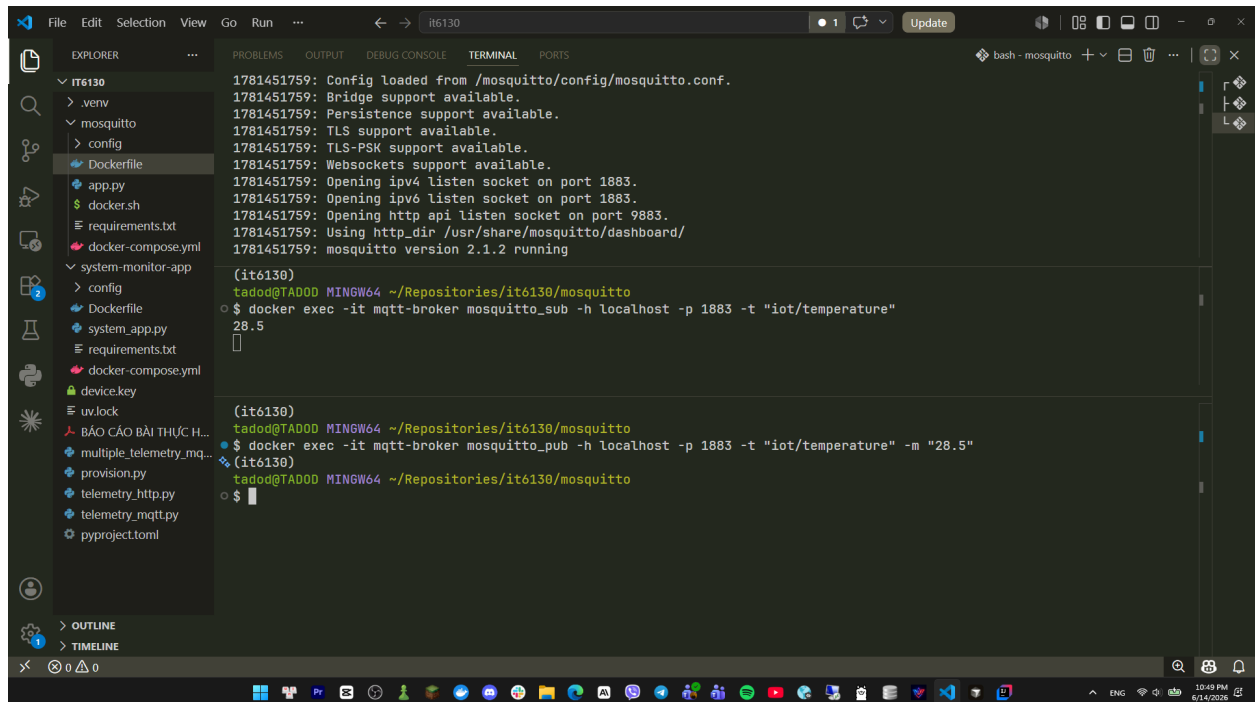


2. Kiểm thử publish/subscribe bằng mosquitto_pub và mosquitto_sub

- Mosquitto_sub với docker exec: `docker exec -it mqtt-broker mosquitto_sub -h localhost -p 1883 -t "iot/temperature"`



- Mosquitto_pub với docker exec và kết quả thu được tại mosquitto_sub: `docker exec -it mqtt-broker mosquitto_pub -h localhost -p 1883 -t "iot/temperature" -m "28.5"`



3. Chạy, hiểu và kiểm thử chương trình [app.py](#) được cung cấp sẵn

```

import json
import os
import random
import time
from datetime import datetime, timezone

import paho.mqtt.client as mqtt

# Thông tin cấu hình
MQTT_BROKER = os.getenv("MQTT_BROKER", "localhost")
MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
MQTT_TOPIC = os.getenv("MQTT_TOPIC", "iot/sensors")
DEVICE_ID = os.getenv("DEVICE_ID", "sensor-01")
PUBLISH_INTERVAL = float(os.getenv("PUBLISH_INTERVAL", "2"))

def create_sensor_data():
    # Generate simulated sensor values.
    temperature = round(random.uniform(25.0, 35.0), 2)
    humidity = round(random.uniform(50.0, 90.0), 2)

```

```

data = {
    "device_id": DEVICE_ID,
    "temperature": temperature,
    "humidity": humidity,
    "timestamp": datetime.now(timezone.utc).isoformat(),
}
return data

def main():
    # Create an MQTT client.
    client = mqtt.Client(client_id=DEVICE_ID)

    print(f"Connecting to MQTT broker at {MQTT_BROKER}:{MQTT_PORT} ...")
    client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)

    print(f"Publishing data to topic: {MQTT_TOPIC}")
    while True:
        data = create_sensor_data()
        payload = json.dumps(data)
        result = client.publish(MQTT_TOPIC, payload)
        if result.rc == mqtt.MQTT_ERR_SUCCESS:
            print(f"Published: {payload}")
        else:
            print(f"Failed to publish message. Error code: {result.rc}")
            time.sleep(PUBLISH_INTERVAL)

if __name__ == "__main__":
    main()

```

- Kết quả chạy app.py và dữ liệu thu được tại mosquitto_sub

```

app.py
1 import json
2 import os
3 import random
4 import time
5 from datetime import datetime, timezone
6
7 import paho.mqtt.client as mqtt
8
9 MQTT_BROKER = os.getenv("MQTT_BROKER", "localhost")
10 MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
11 MQTT_TOPIC = os.getenv("MQTT_TOPIC", "iot/sensors")
12 DEVICE_ID = os.getenv("DEVICE_ID", "sensor-01")
13 PUBLISH_INTERVAL = float(os.getenv("PUBLISH_INTERVAL",
14 "2"))
15
16 def create_sensor_data():
17     # Generate simulated sensor values.
18     temperature = round(random.uniform(25.0, 35.0), 2)
19     humidity = round(random.uniform(50.0, 90.0), 2)
20
21     data = {
22         "device_id": DEVICE_ID,
23         "temperature": temperature,
24         "humidity": humidity,
25         "timestamp": datetime.now(timezone.utc).isoformat()
26     }
27     return data
28
29
30 def main():
31     # Create an MQTT client.
32     client = mqtt.Client(client_id=DEVICE_ID)
33
34     # Connect to the MQTT broker.
35     client.connect(MQTT_BROKER, MQTT_PORT, 60)
36
37     # Subscribe to the MQTT topic.
38     client.subscribe(MQTT_TOPIC)
39
40     # Publish data to the MQTT topic.
41     while True:
42         data = create_sensor_data()
43         client.publish(MQTT_TOPIC, json.dumps(data))
44         time.sleep(PUBLISH_INTERVAL)
45
46 if __name__ == "__main__":
47     main()

```

```

nected.
1781452554: New connection from ::1:55936 on port 1883.
1781452554: New client connected from ::1:55936 as auto-3B823346-FF57-54E0-7B60-C1D3A219F0C5 (p4, c1, k60).
1781452558: New connection from 172.17.0.1:53326 on port 1883.
1781452558: New client connected from 172.17.0.1:53326 as sensor-01 (p4, c1, k60).
[]
(it6130)
tadod@TADOD MINGW64 ~/Repositories/it6130/mosquitto
$ docker exec -it mqtt-broker mosquitto_sub -h localhost -p 1883 -t "iot/sensors"
{"device_id": "sensor-01", "temperature": 31.74, "humidity": 86.26, "timestamp": "2026-06-14T15:55:58.435459+00:00"}
{"device_id": "sensor-01", "temperature": 27.09, "humidity": 89.46, "timestamp": "2026-06-14T15:56:00.436291+00:00"}
[]
(it6130)
tadod@TADOD MINGW64 ~/Repositories/it6130/mosquitto
$ uv run app.py
C:\Users\tadod\Repositories\it6130\mosquitto\app.py:32: DeprecationWarning: Cal
lback API version 1 is deprecated, update to latest version
client = mqtt.Client(client_id=DEVICE_ID)
Connecting to MQTT broker at localhost:1883 ...
Publishing data to topic: iot/sensors
Published: {"device_id": "sensor-01", "temperature": 31.74, "humidity": 86.26,
"timestamp": "2026-06-14T15:55:58.435459+00:00"}
Published: {"device_id": "sensor-01", "temperature": 27.09, "humidity": 89.46,
"timestamp": "2026-06-14T15:56:00.436291+00:00"}

```

4. Container hóa chương trình app.py bằng Dockerfile

- Dockerfile

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]

```

5. Chạy ít nhất 3 sensor simulator container

Để dễ quản lý các container, em sử dụng file docker-compose.yaml để cấu hình toàn bộ các service cần thiết

```

services:
  mqtt-broker:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/config:/mosquitto/config:ro
  # Subscriber nhằm lấy thông tin message thay vì phải chạy terminal sau

```

```
subscriber:
  image: eclipse-mosquitto:latest
  container_name: mqtt-subscriber
  depends_on:
    - mqtt-broker
  command: mosquitto_sub -h mqtt-broker -t iot/sensors -v
  restart: unless-stopped

sensor-01:
  build: .
  container_name: sensor-01
  depends_on:
    - mqtt-broker
  environment:
    MQTT_BROKER: mqtt-broker
    MQTT_PORT: "1883"
    MQTT_TOPIC: iot/sensors
    DEVICE_ID: sensor-01
    PUBLISH_INTERVAL: "2"
  restart: unless-stopped

sensor-02:
  build: .
  container_name: sensor-02
  depends_on:
    - mqtt-broker
  environment:
    MQTT_BROKER: mqtt-broker
    MQTT_PORT: "1883"
    MQTT_TOPIC: iot/sensors
    DEVICE_ID: sensor-02
    PUBLISH_INTERVAL: "3"
  restart: unless-stopped

sensor-03:
  build: .
  container_name: sensor-03
  depends_on:
```

```

- mqtt-broker
environment:
  MQTT_BROKER: mqtt-broker
  MQTT_PORT: "1883"
  MQTT_TOPIC: iot/sensors
  DEVICE_ID: sensor-03
  PUBLISH_INTERVAL: "5"
restart: unless-stopped

```

- Chạy toàn bộ với docker compose up

The screenshot shows the VS Code interface with the `docker-compose.yml` file open. The file defines three services: `mqtt-broker`, `subscriber`, and two sensors (`sensor-01` and `sensor-02`). The `mqtt-broker` service uses the `eclipse-mosquitto:latest` image and is configured with `MQTT_BROKER: mqtt-broker`, `MQTT_PORT: "1883"`, `MQTT_TOPIC: iot/sensors`, `DEVICE_ID: sensor-01`, and `PUBLISH_INTERVAL: "2"`. The `subscriber` service also uses `eclipse-mosquitto:latest` and is configured with `MQTT_BROKER: mqtt-broker`, `MQTT_PORT: "1883"`, `MQTT_TOPIC: iot/sensors`, `DEVICE_ID: sensor-01`, and `PUBLISH_INTERVAL: "2"`. The `sensor-01` and `sensor-02` services are defined with `build: .` and `depends_on: - mqtt-broker`.

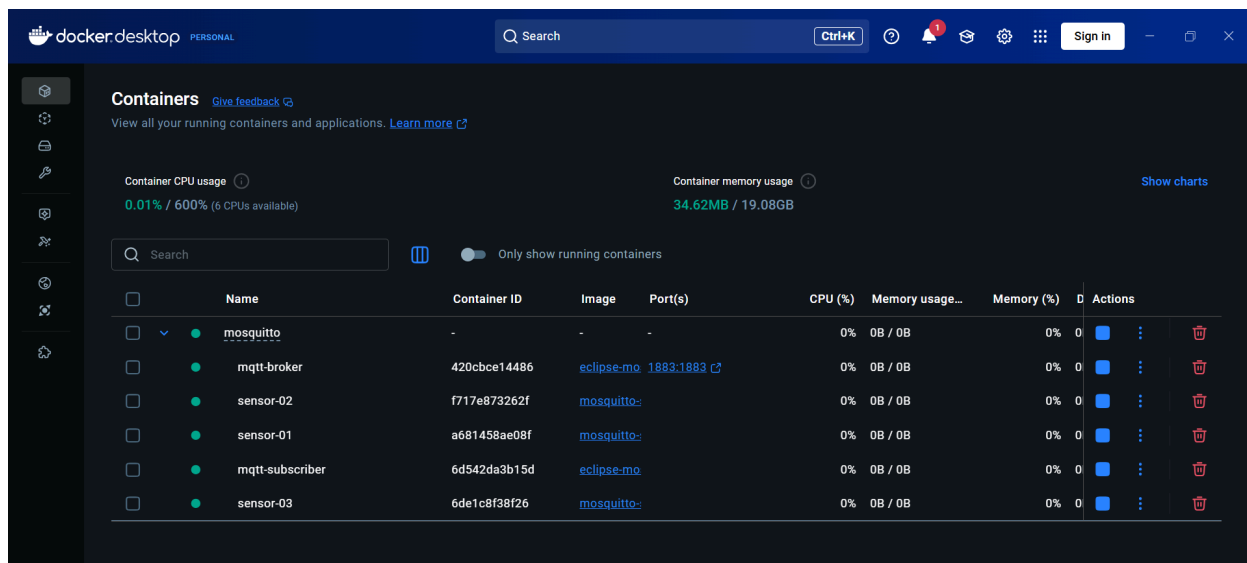
The terminal output shows the command `docker compose up -d` being executed, resulting in the following output:

```

(it6130)
tadod@TAD00 MINGW64 ~/Repositories/it6130/mosquitto
$ docker compose up -d
[+] Running 6/6
  ✓ Network mosquitto_default      Created           0.0s
  ✓ Container mqtt-broker          Started           1.0s
  ✓ Container sensor-03            Started           1.1s
  ✓ Container sensor-01            Started           1.3s
  ✓ Container mqtt-subscriber      Started           1.3s
  ✓ Container sensor-02            Started           1.2s

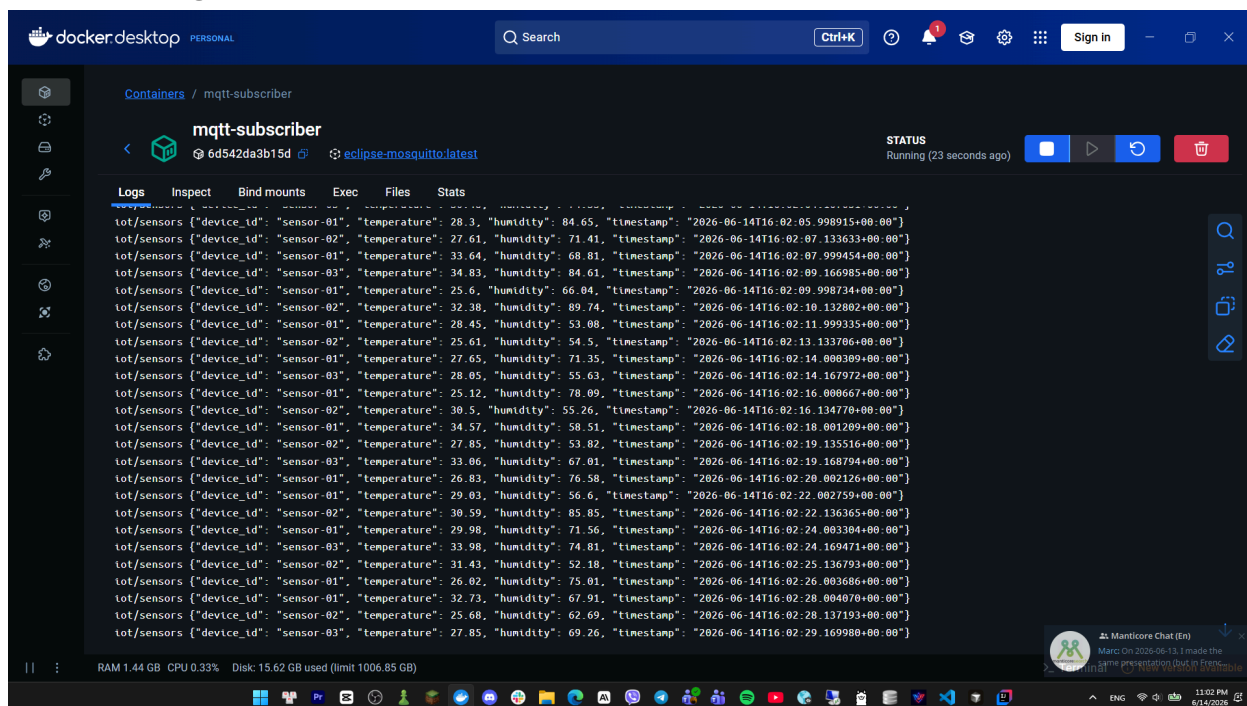
```

- Các container hiển thị trên Docker Desktop



6. Subscribe topic `iot/sensors` và chụp màn hình dữ liệu nhận được

- Thông tin nhận được từ container `mqtt-subscriber`



7. Tự viết một ứng dụng IoT riêng, ví dụ ứng dụng gửi thông tin CPU/RAM lên topic `system/info`

Code ứng dụng:

```
import json
import os
```



```

import time
from datetime import datetime, timezone

import paho.mqtt.client as mqtt
import psutil # Lấy thông tin hệ thống

MQTT_BROKER = os.getenv("MQTT_BROKER", "localhost")
MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))
MQTT_TOPIC = os.getenv("MQTT_TOPIC", "system/info")
DEVICE_ID = os.getenv("DEVICE_ID", "system-monitor-01")
PUBLISH_INTERVAL = float(os.getenv("PUBLISH_INTERVAL", "2"))

def create_system_data():
    # Collect real system metrics via psutil.
    mem = psutil.virtual_memory()
    disk = psutil.disk_usage("/")

    data = {
        "device_id": DEVICE_ID,
        # cpu_percent(interval=None) reports usage since the previous call,
        # so it is primed once in main() before the loop starts.
        "cpu_percent": psutil.cpu_percent(interval=None),
        "cpu_count": psutil.cpu_count(),
        "ram_percent": mem.percent,
        "ram_used_mb": round(mem.used / (1024 * 1024), 2),
        "ram_total_mb": round(mem.total / (1024 * 1024), 2),
        "disk_percent": disk.percent,
        "disk_used_gb": round(disk.used / (1024 ** 3), 2),
        "disk_total_gb": round(disk.total / (1024 ** 3), 2),
        "boot_time": datetime.fromtimestamp(
            psutil.boot_time(), timezone.utc
        ).isoformat(),
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }
    return data

def main():

```

```

# Create an MQTT client.
client = mqtt.Client(client_id=DEVICE_ID)

print(f"Connecting to MQTT broker at {MQTT_BROKER}:{MQTT_PORT} ...")
client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)

# Prime cpu_percent so the first reading in the loop is meaningful
# (the first call always returns 0.0).
psutil.cpu_percent(interval=None)

print(f"Publishing data to topic: {MQTT_TOPIC}")
while True:
    data = create_system_data()
    payload = json.dumps(data)
    result = client.publish(MQTT_TOPIC, payload)
    if result.rc == mqtt.MQTT_ERR_SUCCESS:
        print(f"Published: {payload}")
    else:
        print(f"Failed to publish message. Error code: {result.rc}")
        time.sleep(PUBLISH_INTERVAL)
if __name__ == "__main__":
    main()

```

8. Tự docker hóa ứng dụng IoT riêng đó và chạy trong Docker network iot-net

- Cấu hình file Dockerfile

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY system_app.py .
CMD ["python", "system_app.py"]

```

- File docker-compose để cấu hình số lượng container và subscriber topic với network iot-net

```

services:
  mqtt-broker:
    image: eclipse-mosquitto:latest
    container_name: system-mqtt-broker

```

```

ports:
  - "1884:1883"
volumes:
  - ./config:/mosquitto/config:ro
networks:
  - iot-net

subscriber:
  image: eclipse-mosquitto:latest
  container_name: system-mqtt-subscriber
  depends_on:
    - mqtt-broker
  command: mosquitto_sub -h mqtt-broker -t system/info -v
  restart: unless-stopped
  networks:
    - iot-net

system-monitor-01:
  build: .
  container_name: system-monitor-01
  depends_on:
    - mqtt-broker
  environment:
    MQTT_BROKER: mqtt-broker
    MQTT_PORT: "1883"
    MQTT_TOPIC: system/info
    DEVICE_ID: system-monitor-01
    PUBLISH_INTERVAL: "2"
  restart: unless-stopped
  networks:
    - iot-net

networks:
  iot-net:
    name: iot-net

```

- Chạy ứng dụng cùng broker và subscriber với docker compose up

The screenshot shows a VS Code editor with a project named 'it6130'. The file explorer on the left shows a directory structure including 'system-monitor-app' and 'docker-compose.yml'. The 'docker-compose.yml' file is open in the editor, showing the following configuration:

```

services:
  mqtt-broker:
    image: eclipse-mosquitto:latest
    container_name: system-mqtt-broker
    ports:
      - "1884:1883"
    volumes:
      - ./config:/mosquitto/config:ro
    networks:
      - iot-net

  subscriber:
    image: eclipse-mosquitto:latest
    container_name: system-mqtt-subscriber
    depends_on:
      - mqtt-broker
    command: mosquitto_sub -h mqtt-broker -t system/info -v
    restart: unless-stopped
    networks:
      - iot-net

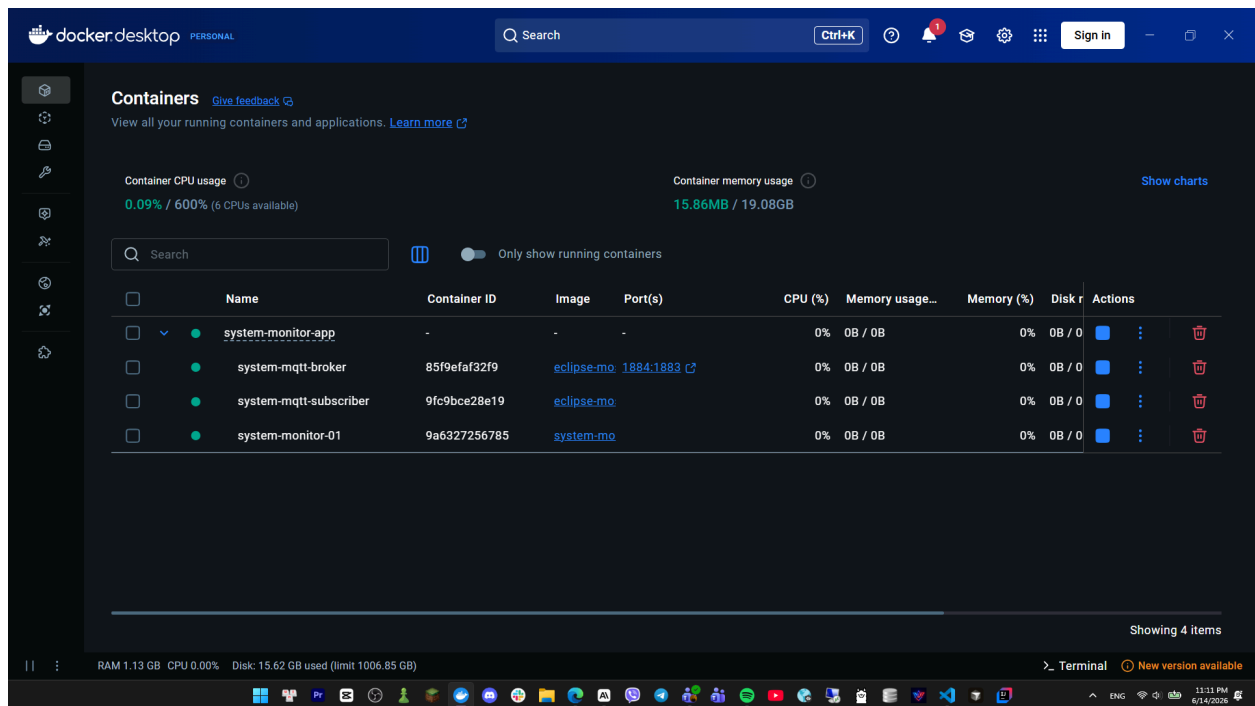
  system-monitor-01:
    build: .
    container_name: system-monitor-01
    depends_on:
      - mqtt-broker
    environment:
      MQTT_BROKER: mqtt-broker
      MQTT_PORT: "1883"
      MQTT_TOPIC: system/info
      DEVICE_ID: system-monitor-01
      PUBLISH_INTERVAL: "2"
    restart: unless-stopped
  
```

The terminal on the right shows the command `docker compose up -d` being executed, resulting in the following output:

```

(it6130)
tadod@TAD00 MINGW64 ~/Repositories/it6130/system-monitor-app
$ docker compose up -d
[+] Running 4/4
  ✓ Network iot-net                      Creat...           0.0s
  ✓ Container system-mqtt-broker         Started           0.4s
  ✓ Container system-mqtt-subscriber     Started           0.7s
  ✓ Container system-monitor-01         Started           0.6s
$
  
```

- **Hiện thị container trên Docker Desktop**



- **Kiểm tra tên network**

The screenshot shows a VS Code editor with a file explorer on the left displaying the project structure for 'it6130'. The main editor shows the 'docker-compose.yml' file with the following content:

```

services:
  mqtt-broker:
    image: eclipse-mosquitto:latest
    container_name: system-mqtt-broker
    ports:
      - "1884:1883"
    volumes:
      - ./config/mosquitto/config:ro
    networks:
      - iot-net

  subscriber:
    image: eclipse-mosquitto:latest
    container_name: system-mqtt-subscriber
    depends_on:
      - mqtt-broker
    command: mosquitto_sub -h mqtt-broker -t system/
    info -v
    restart: unless-stopped
    networks:
      - iot-net

system-monitor-01:
  build: .
  container_name: system-monitor-01
  depends_on:
    - mqtt-broker
  environment:
    MQTT_BROKER: mqtt-broker
    MQTT_PORT: "1883"
    MQTT_TOPIC: system/info
    DEVICE_ID: system-monitor-01
    PUBLISH_INTERVAL: "2"
  restart: unless-stopped

```

The terminal window on the right shows the command 'docker compose up -d' being executed, resulting in the following output:

```

(it6130)
tadod@TAD00 MINGW64 ~/Repositories/it6130/system-monitor-app
$ docker compose up -d
[+] Running 4/4
  ✓ Network iot-net                  Creat...           0.0s
  ✓ Container system-mqtt-broker     Started           0.4s
  ✓ Container system-mqtt-subscriber Started           0.7s
  ✓ Container system-monitor-01     Started           0.6s
(it6130)
tadod@TAD00 MINGW64 ~/Repositories/it6130/system-monitor-app
$ docker network ls

```

The 'docker network ls' command output is as follows:

NETWORK ID	NAME	DRIVER	SCOPE
3f27df3ce131	bridge	bridge	local
a54e945d22e1	host	host	local
ed6be05e0ffa	iot-net	bridge	local
8bbf722286c9	none	null	local
bf01cc3f0bf	system-monitor-app_default	bridge	local

9. Subscribe topic riêng của ứng dụng mới và chụp màn hình dữ liệu nhận được

Do đã có container riêng cho subscribe to topic, dữ liệu nhận được có thể check qua Docker Desktop

The screenshot shows the Docker Desktop interface. The 'Containers' tab is selected, and the 'system-mqtt-subscriber' container is highlighted. The 'Logs' tab is active, displaying the following log entries:

```

{"disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:12:48.869946+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.1, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 725.55, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:12:50.870716+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.2, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 725.55, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:12:52.871442+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 1.7, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 719.5, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:12:54.872312+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.6, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 717.13, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:12:56.873188+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.0, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 717.13, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:12:58.874217+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.0, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 717.13, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:13:00.874829+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.1, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 717.13, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:13:02.875595+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.8, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 717.38, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:13:04.876556+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.4, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 722.44, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:13:06.877471+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.0, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 722.44, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:13:08.876306+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.2, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 722.38, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:13:10.877055+00:00"}
system/info {"device_id": "system-monitor-01", "cpu_percent": 0.2, "cpu_count": 6, "ram_percent": 5.1, "ram_used_mb": 722.37, "ram_total_mb": 20006.44, "disk_percent": 1.6, "disk_used_gb": 15.62, "disk_total_gb": 1006.85, "boot_time": "2026-06-14T15:37:32+00:00", "timestamp": "2026-06-14T16:13:12.877801+00:00"}

```

10. Trả lời các câu hỏi ở Mục 14

10.1. MQTT broker có vai trò gì trong hệ thống IoT?

MQTT broker là thành phần trung gian nhận message từ các publisher và chuyển tiếp đến các subscriber theo đúng topic. Trong IoT, broker giúp các thiết bị (cảm biến, ứng dụng) trao đổi dữ liệu một cách gián tiếp, không cần kết nối trực tiếp với nhau, nhờ đó hệ thống dễ mở rộng và quản lý hơn.

10.2. Sự khác nhau giữa MQTT publisher và MQTT subscriber là gì?

Publisher là client gửi message lên một topic. Subscriber là client đăng ký nhận message từ một topic mà nó quan tâm. Một client có thể vừa là publisher vừa là subscriber. Cả hai đều giao tiếp thông qua broker chứ không trực tiếp với nhau.

10.3. Vì sao cần dùng Docker container để đóng gói ứng dụng IoT?

Vì container đóng gói ứng dụng cùng toàn bộ thư viện và dependency cần thiết, giúp ứng dụng chạy nhất quán trên mọi môi trường (máy lập trình viên, server, máy ảo). Nó cô lập ứng dụng, dễ triển khai, dễ nhân bản nhiều thiết bị, và tránh xung đột phiên bản thư viện giữa các ứng dụng.

10.4. Trong lệnh chạy broker, tham số -p 1883:1883 có ý nghĩa gì?

Nó ánh xạ (map) cổng 1883 của máy host vào cổng 1883 của container. Nhờ đó các client bên ngoài container (ví dụ mosquitto_sub chạy trên host) có thể kết nối tới broker qua localhost:1883. Cú pháp -p <host_port>:<container_port>.

10.5. Trong lệnh chạy sensor container, vì sao MQTT_BROKER được đặt là mqtt-broker thay vì localhost?

Vì khi chạy trong cùng một Docker network, mỗi container có địa chỉ mạng riêng. localhost bên trong container sensor chỉ trỏ về chính container sensor đó, không phải broker. Khi đặt là mqtt-broker (tên container), Docker sẽ tự phân giải tên này thành IP của container broker, giúp sensor kết nối đúng tới broker.

10.6. Docker network `iot-net` có vai trò gì?

Đây là một mạng ảo nội bộ cho phép các container nằm chung mạng giao tiếp với nhau bằng tên container thay vì địa chỉ IP. Nhờ `iot-net`, sensor có thể gọi broker qua tên `mqtt-broker`, giúp cấu hình đơn giản và ổn định hơn.

10.7. Nếu không dùng Docker network riêng, sensor container có thể kết nối đến broker bằng tên `mqtt-broker` không? Giải thích.

Không. Trên mạng bridge mặc định của Docker, cơ chế phân giải tên container (DNS nội bộ) không hoạt động, nên sensor sẽ không tìm được broker qua tên `mqtt-broker`. Tính năng phân giải tên chỉ có khi các container cùng nằm trong một user-defined network như `iot-net`. (Nếu không, phải dùng IP của broker hoặc các cách khác như `--link` đã lỗi thời.)

10.8. Vì sao chương trình Python nên đọc cấu hình từ biến môi trường thay vì hard-code trong source code?

Để tăng tính linh hoạt và tái sử dụng: có thể thay đổi broker, port, topic, device ID, chu kỳ gửi... mà không cần sửa và build lại code. Điều này phù hợp với mô hình container (cấu hình tách khỏi image), giúp dùng chung một image cho nhiều thiết bị với cấu hình khác nhau, đồng thời tránh lộ thông tin nhạy cảm trong source.

10.9. Nếu muốn thay đổi chu kỳ gửi dữ liệu từ 2 giây sang 10 giây mà không sửa code, cần làm gì?

Chỉ cần đặt lại biến môi trường `PUBLISH_INTERVAL=10` khi chạy container, ví dụ thêm tham số `-e PUBLISH_INTERVAL=10` trong lệnh `docker run` (hoặc sửa giá trị trong file `docker-compose`). Không cần sửa hay build lại code.

10.10. Trong hệ thống IoT thật, vì sao không nên để `allow_anonymous true` cho MQTT broker?

Vì nó cho phép bất kỳ client nào kết nối mà không cần xác thực, dẫn đến rủi ro bảo mật nghiêm trọng: kẻ xấu có thể đọc trộm dữ liệu cảm biến, gửi dữ liệu giả mạo, hoặc tấn công hệ thống. Hệ thống thật cần bật xác thực bằng username/password (và nên dùng TLS) để kiểm soát truy cập.

10.11. Ứng dụng IoT riêng của bạn publish dữ liệu lên topic nào? Vì sao bạn chọn topic đó?

Ứng dụng publish lên topic system/info. Chọn topic này vì nó mô tả đúng bản chất dữ liệu (thông tin hệ thống), có cấu trúc phân cấp rõ ràng theo nhánh system/, dễ mở rộng thành các topic con như system/cpu, system/ram, và thuận tiện cho việc subscribe bằng wildcard system/#.

10.12. Ứng dụng IoT riêng của bạn gửi những trường dữ liệu nào trong JSON message?

Các trường tối thiểu gồm: device_id, cpu_percent, ram_percent, và timestamp. (Có thể bổ sung thêm disk_percent, hostname tùy hướng mở rộng.)

10.13. Vai trò của thư viện psutil là gì trong ứng dụng gửi thông tin hệ thống?

psutil là thư viện Python dùng để lấy thông tin tài nguyên hệ thống như mức sử dụng CPU, RAM, dung lượng đĩa, tiến trình, mạng... Trong ứng dụng này, nó cung cấp các giá trị cpu_percent, ram_percent, disk_percent để đóng gói thành dữ liệu JSON gửi lên broker.

10.14. Khi chạy ứng dụng IoT riêng trong container, vì sao vẫn cần đặt MQTT_BROKER=mqtt-broker?

Vì ứng dụng chạy trong container riêng, nó cần biết địa chỉ của broker trên mạng iot-net. Đặt MQTT_BROKER=mqtt-broker để Docker phân giải tên này thành IP của container broker, giúp ứng dụng kết nối đúng. Nếu để localhost, ứng dụng sẽ tìm broker ngay trong chính container của nó và không kết nối được.